

RTL Specifications

Sequence Detector · Traffic Light Controller · Synchronous FIFO

Asmicore Semiconductor Internship — 14-Day RTL/DV Assignment, Day 8

Author: Nur Mohammad Abdullah

All three designs verified via Icarus Verilog v12.0 simulation; see docs/experiments/ for logs.

RTL #1 — Sequence Detector (Overlapping “1011”)

Overview

Detects the overlapping bit pattern “1011” arriving serially, one bit per clock cycle, on a single-bit input. “Overlapping” means the trailing bit of a completed match may also serve as the first bit of the next match (e.g. the stream “1011011” contains two occurrences of “1011”, sharing one bit between them) — the detector must correctly fire on both, not just the first. Implemented as a **genuine 5-state Moore FSM**: detected is a pure combinational decode of the current state (`detected = (state == S1011)`), HIGH only while the FSM occupies the dedicated DETECT state `s1011`. Because it depends on the registered state alone — never on `din` — it is glitch-free on the input and unambiguously a Moore output.

Key Features

- Overlapping detection — correctly handles matches that share bits with the preceding match.
- True Moore output: `detected` is a pure function of state (HIGH only in the DETECT state `s1011`), with no combinational dependence on `din`.
- Asynchronous, active-low reset (`rst_n`) — guarantees a known state independent of clock activity.
- 5-state FSM, 3-bit encoding: `IDLE`, `S1`, `S10`, `S101`, `S1011`.

Interface Table

Port	Direction	Width	Clock Domain	Active Level	Description
clk	input	1	— (source)	rising-edge	System clock. All sequential logic (the state register) updates on the rising edge.
rst_n	input	1	clk	active-low	Asynchronous reset. Forces the FSM to <code>IDLE</code> (so <code>detected=0</code>), independent of <code>clk</code> activity.
din	input	1	clk	active-high, sampled	Serial data input bit, sampled once per rising edge of <code>clk</code> . The FSM tracks how much of “1011” has been matched using this bit.
detected	output	1	clk	active-high, one-cycle pulse	Pulses HIGH for exactly one cycle when the FSM confirms <code>din</code> has completed an occurrence of “1011”. A Moore decode of state — <code>detected = (state == S1011)</code> — so the pulse is HIGH the cycle after the 4th matching bit (in state <code>s1011</code>); confirmed via simulation (see Timing).

Microarchitecture

Implemented as **two** `always` **blocks** plus a **combinational output assign**, each with a single responsibility:

- **Block 1 — State register** (`always_ff`, `async reset`): the only place the state variable is updated.
- **Block 2 — Next-state logic** (`always_comb`): purely combinational; computes the next state from the current state and `din`. Adds the DETECT state `s1011`, entered from `s101` on `din=1`, and handled like `s1` on the following input so overlap works.
- **Output** (`assign`): `detected = (state == S1011)`. Because it decodes the registered state, it changes only when the state changes (on a clock edge) and never depends on `din` — the defining property of a Moore output. No separate output register is needed.

FSM Specification

State Encoding

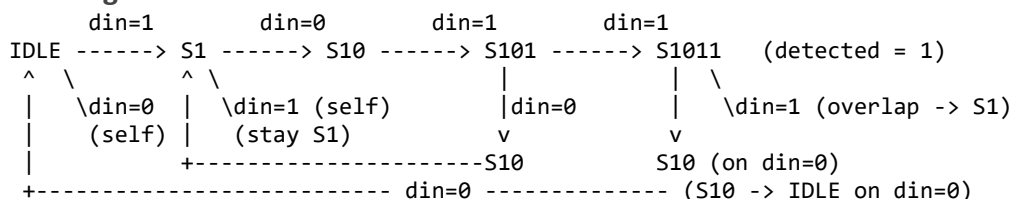
State	Encoding	Description
IDLE	3'b000	No useful progress made yet toward matching “1011”.
S1	3'b001	Most recent bit(s) match the prefix “1”.
S10	3'b010	Most recent bits match the prefix “10”.
S101	3'b011	Most recent bits match the prefix “101” — one more ‘1’ completes the match.
S1011	3'b100	Full match just completed — DETECT state; <code>detected</code> is HIGH here only.

Transition Table

Current State	Input (din)	Next State	Output (detected)
IDLE	0	IDLE	0
IDLE	1	S1	0
S1	0	S10	0
S1	1	S1	0
S10	0	IDLE	0
S10	1	S101	0
S101	0	S10	0
S101	1	S1011	0 (this edge)
S1011	0	S10	1 (in S1011)
S1011	1	S1	1 (in S1011; overlap)

`detected` is a pure Moore output: it is HIGH exactly while the FSM is in `s1011`, regardless of `din`.

State Diagram



- `S101 --din=1--> S1011` completes the match; `detected` is HIGH while in `s1011`.

- $s_{1011} \xrightarrow{\text{din}=1} s_1$ is the overlap: the trailing ‘1’ of a completed match immediately seeds the next potential match.
- Both s_{101} and s_{1011} fall back to s_{10} on $\text{din}=0$ (the “...10” tail is still useful progress).

Timing

Actual simulation trace (Icarus Verilog v12.0), stimulus vector $\text{din} = 1,1,0,1,0,1,1$ — the embedded “1011” is completed by the 7th input bit. Each row shows the state the FSM is **in during** that cycle (pre-edge) and detected for that cycle:

Cycle	din (consumed)	state (this cycle)	detected
4	1	s_{10}	0
5	0	s_{101}	0
6	1	s_{10}	0
7	1	s_{101}	0
8	0	s_{1011}	1
9	0	s_{10}	0

detected fires in cycle 8, while the FSM is in the DETECT state s_{1011} . The 4th matching bit is applied in cycle 7 (state still s_{101} , *detected* still 0); the FSM enters s_{1011} on that edge and the pulse reads HIGH the following cycle — the ordinary one-cycle latency of a clocked FSM output. An intermediate draft mislabeled this as “cycle 7, no latency”; that was a post-edge print artifact, corrected — see ISSUE-D4-01 in the Issue Log.

Corner Cases

#	Corner Case	Expected Behavior
1	Overlapping match immediately after a previous match (e.g. ...1011011...)	Both occurrences of “1011” must be detected. Confirmed by the dedicated self-checking overlap regression (<code>seq_detect_1011_overlap_tb</code>) driving 1011011: two detections fire, in cycles 5 and 8 (3-cycle gap, the minimum for overlapping “1011”). Note: the Day-4 smoke vector 1101011 contains only one “1011”, so a dedicated overlap test is required.
2	Reset asserted mid-sequence (e.g. FSM is in s_{101})	FSM must return cleanly to IDLE, <i>detected</i> must go LOW, partial match progress discarded.
3	din held constant (all 0s or all 1s) for many cycles	Must never falsely assert <i>detected</i> ; must self-loop correctly (IDLE on all-0s, s_1 on all-1s) indefinitely.
4	First bit after reset is a 1	FSM must immediately begin progressing from IDLE — no warm-up cycle required.

Verification Plan

Built during the Day 12 traceability audit — the Days 9–11 verification-planning exercises were originally scoped only to the FIFO; this table closes that gap for the Sequence Detector.

Feature	Req-ID	Cat.	Checker	Stimulus	Pr i.	Coverage Item
reset_returns_fsm_to_idle_and_detected_low	RST_001	RST	Expected value	Directed	P0	Coverpoint (state,detected) post-

Feature	Req-ID	Cat.	Checker	Stimulus	Pr i.	Coverage Item
						reset; bin {IDLE,0}
basic_sequence_detected_correctly	FUNC_001	FUNC	Expected value	Directed	P0	Coverpoint detected bin fired={1}; coverpoint state, all 5 bins
overlapping_matches_both_detected	CORNER_001	CORNER	Expected value	Directed	P0	3-cycle overlap-gap bin (design's literal purpose, not just a corner case)
reset_mid_sequence_discards_partial_progress	CORNER_002	CORNER	Expected value	Directed (from S1/S10/S101)	P0	Cross: state-at-reset × state-after-reset
constant_din_never_falsely_detects	STRESS_001	STRESS	Assertion	Directed, long run	P1	Coverpoint on constant-din-run length, bin >=20 cycles
first_bit_after_reset_starts_progress_immediately	CORNER_003	CORNER	Expected value	Directed	P2	Coverpoint on first din after reset, bins {0,1}

RTL Code

```
// =====
`timescale 1ns/1ps
// Module      : seq_detect_1011
// Description : Overlapping sequence detector for the bit pattern "1011" on a
//               serial input (din). "Overlapping" means the trailing bit of a
//               completed match may also be the first bit of the next match
//               (e.g. "1011011" fires twice, sharing the middle "1").
//
//               TRUE two-block Moore FSM. 'detected' is a pure function of the
//               CURRENT STATE alone (not of din) - the defining property of a
//               Moore machine. A dedicated DETECT state (S1011) is entered the
//               cycle after the 4th matching bit is sampled, and 'detected' is
//               HIGH exactly while the FSM sits in that state.
//
//               This is the resolution of the Moore-vs-Mealy question raised in
//               the Day 2 state-diagram note: a genuine Moore output for this
//               pattern needs a 5th state, which is added here. (An earlier
//               draft implemented a 4-state Mealy detector with a REGISTERED
//               output - that reproduces Moore *timing* but is not structurally
//               Moore, because its 'detected' is not a function of state alone.
//               See ISSUE-D4-01 in the Issue Log for that history.)
```

```

//
//      Timing: the completing (4th) bit is sampled at a clock edge;
//      the FSM enters S1011 on that edge, so 'detected' reads HIGH
//      during the FOLLOWING cycle - the ordinary one-cycle latency of
//      a clocked FSM. Verified in the Day 4 smoke test.
//
// Author      : Nur
// =====

module seq_detect_1011 (
    input logic clk,      // system clock
    input logic rst_n,    // active-low asynchronous reset
    input logic din,      // serial input bit, one bit sampled per clk cycle
    output logic detected // HIGH for one cycle when "1011" completes (Moore)
);

// -----
// State declaration (5 states -> 3 encoding bits).
// Each state = "longest suffix of the input seen so far that is also a
// prefix of 1011":
// IDLE - ""      : no useful progress yet (reset state)
// S1   - "1"     : matched so far "1"
// S10  - "10"    : matched so far "10"
// S101 - "101"   : matched so far "101" (one more '1' completes it)
// S1011 - "1011" : full match -> DETECT state; 'detected' HIGH here only
// -----
typedef enum logic [2:0] {IDLE, S1, S10, S101, S1011} state_t;
state_t state, next_state;

// -----
// Block 1 - State register (sequential only, nothing else lives here).
// Asynchronous active-low reset: guarantees a known state even before the
// clock is toggling - the safer default for a control FSM.
// -----
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        state <= IDLE;
    else
        state <= next_state;
end

// -----
// Block 2 - Next-state Logic (pure combinational, no registers here).
// Default assignment at the top avoids incomplete-case latch inference.
//
// Overlap handling: after a full match, the trailing '1' of "1011" is a
// fresh 1-bit prefix, so S1011 transitions on the NEXT input exactly like
// S1 does (din=1 -> S1, din=0 -> S10). Both S101 and S1011 fall back to
// S10 on a '0' (the "...10" tail is still useful progress).
// -----
always_comb begin
    next_state = state; // default: hold current state
    case (state)
        IDLE: if (din) next_state = S1;   else next_state = IDLE;
        S1:   if (din) next_state = S1;   else next_state = S10;
        S10:  if (din) next_state = S101; else next_state = IDLE;
        S101: if (din) next_state = S1011; else next_state = S10; // 4th bit '1' -> DETECT
        S1011: if (din) next_state = S1;   else next_state = S10; // overlap: acts like S1
        default: next_state = IDLE; // safety net, unreachable
    endcase
end

// -----
// Moore output: pure function of the current state. HIGH only in the
// dedicated DETECT state, so it never depends on din and cannot glitch on
// din. This is what makes the machine a genuine Moore FSM.

```

```
// -----
assign detected = (state == S1011);

endmodule
```

RTL #2 — Traffic Light Controller

Overview

Cycles a two-road intersection through four timed phases in a fixed round-robin sequence: MAIN_GREEN → MAIN_YELLOW → SIDE_GREEN → SIDE_YELLOW → back to MAIN_GREEN. Unlike the Sequence Detector, this design has no external data input — the only thing that drives a state transition is an internal down-counter reaching zero. This is a direct application of the FSM+datapath pattern: the FSM decides what phase comes next (a fixed sequence), while a separate counter datapath decides how long to stay in each phase.

Key Features

- FSM + datapath pattern — state register and down-counter share one `always_ff` block; next-state logic, duration lookup, and output logic are each separate `always_comb` blocks.
- Parameterized phase durations (default 4, 2, 6, 2 cycles) chosen small so simulation completes quickly.
- Pure Moore output — all six lights are a function of state alone.
- Safe all-red default branch for any unreachable/illegal state value.

Interface Table

Port	Direction	Width	Clock Domain	Active Level	Description
clk	input	1	— (source)	rising-edge	System clock.
rst_n	input	1	clk	active-low	Asynchronous reset. Forces state to MAIN_GREEN and loads the counter with MAIN_GREEN_TIME-1 (the same value a normal reload uses), so the first MAIN_GREEN lasts a full 4 cycles (see ISSUE-D5-02).
main_red	output	1	clk	active-high	Main-road red light.
main_yellow	output	1	clk	active-high	Main-road yellow light.
main_green	output	1	clk	active-high	Main-road green light.
side_red	output	1	clk	active-high	Side-road red light.
side_yellow	output	1	clk	active-high	Side-road yellow light.
side_green	output	1	clk	active-high	Side-road green light.

Microarchitecture

- **State register + down-counter** (`always_ff`): the ONLY sequential block. Load-vs-decrement priority — reaching `count==0` always wins over plain decrementing. At reset it loads `MAIN_GREEN_TIME-1`, identical to a normal reload.
- **Next-state logic** (`always_comb`): a fixed round-robin case statement — answers only “what comes after what,” never “when.”
- **Duration lookup** (`always_comb`): maps whichever state is coming next to that state’s `DURATION-1`, loaded into the counter only at the instant of a reload.
- **Output logic** (`always_comb`): drives all six lights as a pure function of state.

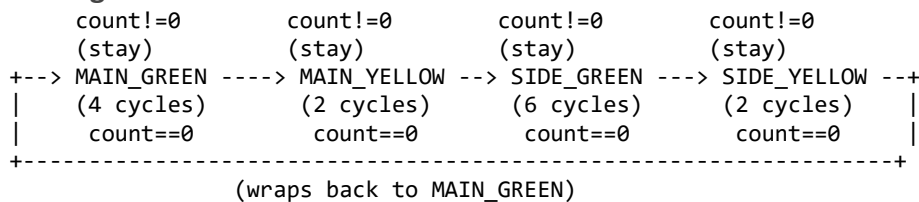
FSM Specification

State + Duration Table

State	Duration (cycles)	main_road lights	side_road lights
MAIN_GREEN	4	green	red
MAIN_YELLOW	2	yellow	red
SIDE_GREEN	6	red	green
SIDE_YELLOW	2	red	yellow

Full loop = 4 + 2 + 6 + 2 = 14 cycles, then repeats. Design decision documented explicitly: during both YELLOW phases, the other road stays RED for the entire phase — there is never a cycle where both roads show anything but “one green/yellow, one red.”

State Diagram



Reset forces `MAIN_GREEN` and loads the counter with `MAIN_GREEN_TIME-1` — **exactly the value a normal reload uses** — so the first `MAIN_GREEN` lasts a full `MAIN_GREEN_TIME` (4) cycles, identical to every later `MAIN_GREEN`. (A D-cycle phase corresponds to the counter stepping `D-1`, `D-2`, ..., `0`, so the reload value is always `D-1`, reset included. An earlier draft loaded the full `MAIN_GREEN_TIME` at reset, making the first phase 5 cycles — see `ISSUE-D5-02`, corrected.) There is no external input anywhere in this diagram — the ONLY thing that drives a transition is the down-counter reaching zero.

Timing

Actual simulation trace (Icarus Verilog v12.0), corrected RTL and honest-sampling testbench (the `count` column is the internal down-counter, exposed so each duration is directly verifiable):

Cycles	State	count steps	Duration observed
1–4	MAIN_GREEN	3→0	4 (matches spec)
5–6	MAIN_YELLOW	1→0	2 (matches spec)
7–12	SIDE_GREEN	5→0	6 (matches spec)
13–14	SIDE_YELLOW	1→0	2 (matches spec)
15–18	MAIN_GREEN (2nd loop)	3→0	4 (confirms correct wraparound)

Cycles	State	count steps	Duration observed
19–20	MAIN_YELLOW (2nd loop, partial)	1→0	sim ends mid-phase

The first and second MAIN_GREEN are both a true 4 cycles, provable from the count column. Note: the reset off-by-one (ISSUE-D5-02) was originally hidden because the first smoke testbench dropped the reset cycle from its printout (ISSUE-D5-03), so a 5-cycle first phase printed as 4 and matched the later 4-cycle phase. Honest per-cycle sampling plus the exposed count is what made it visible — a two-loop check alone did not catch it.

Corner Cases

#	Corner Case	Expected Behavior
1	Reset asserted mid-countdown in a non-initial state	Must always return to MAIN_GREEN and reload the counter with MAIN_GREEN_TIME-1, giving a full 4-cycle MAIN_GREEN regardless of which state/count reset caught the FSM at.
2	Wraparound from SIDE_YELLOW back to MAIN_GREEN	Must reload for a full duration every time — not just the first. The 20-cycle smoke test (one full loop plus 6 cycles) shows first and second MAIN_GREEN both a true 4 cycles via the count column. (Historical note: the mis-sampled smoke log masked ISSUE-D5-02 rather than catching it — see ISSUE-D5-03.)
3	Duration parameter set to the minimum realistic value (1 cycle)	Load-vs-decrement priority must still advance exactly one cycle later. Confirmed by the min-duration regression (traffic_light_min_duration_tb, MAIN_YELLOW_TIME=1): MAIN_YELLOW occupies exactly one cycle (count loaded to 0, reloads next edge) — closing ISSUE-D7-01.
4	Two adjacent states sharing the same duration value	Counter reaching zero must trigger the correct next state per the fixed sequence, regardless of duration coincidence.

Verification Plan

Built during the Day 12 traceability audit. RST_001 below is the permanent regression test for ISSUE-D5-02 — the actual reset bug found on Day 5.

Feature	Req-ID	Cat.	Checker	Stimulus	Pr i.	Coverage Item
reset_forces_main_green_full_4_cycles	RST_001	RST	Expected value	Directed	P0	Coverpoint (state,count) post-reset; bin confirms count==MAIN_GREEN_TIME-1 loaded (ISSUE-D5-02 regression)
full_cycle_sequence_correct	FUNC_001	FUNC	Expected value	Directed	P0	State+duration coverpoint, all 4 states × correct hold-length
wraparound_reloads_full_duration_every_time	CORNER_001	CORNER	Expected value	Directed, 2+ loops	P0	Cross: loop-iteration × MAIN_GREEN duration, compared numerically against count (prevents the

Feature	Req-ID	Cat.	Checker	Stimulus	Pr i.	Coverage Item
						ISSUE-D5-02 class of regression)
mutual_exclusion_of_green_lights	CORNER_002	CORNER	Assertion	N/A (free-running)	P0	Cross (main_green,side_green); bin {1,1} must NEVER be hit
yellow_phase_other_road_stays_red	CORNER_003	CORNER	Assertion	N/A	P1	Cross (main_yellow,side_red) and (side_yellow,main_red)
one_hot_lights_per_road	CORNER_004	CORNER	Assertion	N/A	P1	Per-road coverpoint (likely redundant with FUNC_001 — noted, not hidden)

RTL Code

```
// =====
`timescale 1ns/1ps
// Module      : traffic_light
// Description : Traffic light controller cycling through four timed phases:
//              MAIN_GREEN -> MAIN_YELLOW -> SIDE_GREEN -> SIDE_YELLOW -> (repeat)
//              Durations are parameterized (small default values chosen so
//              the smoke test finishes quickly in simulation - see Day 5
//              Learning Topic 5.2, "parameter for duration values - makes
//              the design testable with short durations").
//
//              FSM + datapath pattern (Learning Topic 5.1): the FSM decides
//              WHAT phase comes next (a fixed round-robin sequence - there
//              is no external input driving this design at all); a
//              down-counter datapath decides HOW LONG to stay in each
//              phase. The FSM only advances when the counter reaches zero.
//
// Author       : Nur (RTL implemented as part of Day 5 experimental work)
// =====

module traffic_light #(
    parameter int MAIN_GREEN_TIME  = 4, // cycles to hold MAIN_GREEN
    parameter int MAIN_YELLOW_TIME = 2, // cycles to hold MAIN_YELLOW
    parameter int SIDE_GREEN_TIME  = 6, // cycles to hold SIDE_GREEN
    parameter int SIDE_YELLOW_TIME = 2, // cycles to hold SIDE_YELLOW
    // Must be >= the largest of the four durations above. Only this one
    // parameter needs widening if you increase any duration beyond 6 -
    // it sizes the counter, nothing else.
    parameter int MAX_DURATION     = 6
) (
    input  logic clk,
    input  logic rst_n,           // active-low asynchronous reset

    output logic main_red,
    output logic main_yellow,
    output logic main_green,
    output logic side_red,
    output logic side_yellow,
    output logic side_green
);
```

```

// -----
// State declaration - four phases, fixed round-robin sequence.
// Unlike Day 4's sequence detector, there is no external input here:
// the ONLY thing that ever causes a transition is the counter
// reaching zero.
// -----
typedef enum logic [1:0] {
    MAIN_GREEN, MAIN_YELLOW, SIDE_GREEN, SIDE_YELLOW
} state_t;
state_t state, next_state;

localparam int CNT_WIDTH = $clog2(MAX_DURATION);
logic [CNT_WIDTH-1:0] count;           // down-counter: current phase's remaining cycles
logic [CNT_WIDTH-1:0] next_duration_m1; // duration-1 for whichever phase comes next

// -----
// Next-state logic (always_comb) - purely the fixed sequence. This
// block only answers "what comes after what," never "when" - timing
// is entirely the counter's job, kept in a separate block on purpose
// (Learning Topic 5.1: "FSM controls state, counter controls duration").
// -----
always_comb begin
    case (state)
        MAIN_GREEN: next_state = MAIN_YELLOW;
        MAIN_YELLOW: next_state = SIDE_GREEN;
        SIDE_GREEN: next_state = SIDE_YELLOW;
        SIDE_YELLOW: next_state = MAIN_GREEN;
        default:     next_state = MAIN_GREEN; // safety net, unreachable
    endcase
end

// Duration-minus-1 for whichever state we are ABOUT to enter (used only
// at the instant the counter gets reloaded, in the always_ff below).
// Minus 1 because the reload cycle itself is the first of that phase's
// 'duration' cycles - e.g. MAIN_GREEN_TIME=4 means 4 total cycles in
// MAIN_GREEN, so the counter counts 3,2,1,0 (4 values).
always_comb begin
    case (next_state)
        MAIN_GREEN: next_duration_m1 = MAIN_GREEN_TIME - 1;
        MAIN_YELLOW: next_duration_m1 = MAIN_YELLOW_TIME - 1;
        SIDE_GREEN: next_duration_m1 = SIDE_GREEN_TIME - 1;
        SIDE_YELLOW: next_duration_m1 = SIDE_YELLOW_TIME - 1;
        default:     next_duration_m1 = MAIN_GREEN_TIME - 1;
    endcase
end

// -----
// State register + down-counter, in ONE always_ff, per the assignment.
// Load-vs-decrement priority: reaching zero (i.e. "done") always wins
// over plain decrementing, since that is the one event that means
// "this phase is over, move on and reload for the next one."
// -----
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state <= MAIN_GREEN;
        count <= MAIN_GREEN_TIME - 1; // -1 so the FIRST MAIN_GREEN lasts exactly
                                      // MAIN_GREEN_TIME cycles, identical to every
                                      // later MAIN_GREEN (see ISSUE-D5-02, corrected)
    end else if (count == '0) begin
        state <= next_state;
        count <= next_duration_m1;
    end else begin
        count <= count - 1'b1;
    end
end
end

```

```
// -----
// Output Logic (always_comb) - pure Moore output, a function of state
// alone. One-hot-per-road-pair style: exactly one of {red, yellow,
// green} is HIGH per road at all times.
//
// Explicit design decision (documented, not left implicit - see Day 7's
// "don't leave it implicit" principle): during BOTH yellow phases, the
// *other* road is RED for the entire phase. There is never a moment
// where both roads show anything other than "one green/yellow, one
// red."
// -----
always_comb begin
    {main_red, main_yellow, main_green, side_red, side_yellow, side_green} = '0;
    case (state)
        MAIN_GREEN:    begin main_green = 1'b1; side_red    = 1'b1; end
        MAIN_YELLOW:   begin main_yellow = 1'b1; side_red    = 1'b1; end
        SIDE_GREEN:    begin main_red    = 1'b1; side_green  = 1'b1; end
        SIDE_YELLOW:   begin main_red    = 1'b1; side_yellow = 1'b1; end
        default:       begin main_red    = 1'b1; side_red    = 1'b1; end // safe all-red default
    endcase
end

endmodule
```

RTL #3 — Synchronous FIFO

Overview

This is a standard single-clock-domain synchronous FIFO, 8 entries deep and 8 bits wide (both parameterizable). It has the usual write side (`wr_en`, `din`) and read side (`rd_en`, `dout`), plus `full` and `empty` status flags. Internally it's built from an 8-entry memory array plus a write pointer, a read pointer, and a count register — `full` and `empty` key off that count register specifically, because comparing the two pointers directly can't tell you whether the FIFO is completely empty or completely full (both conditions make the pointers equal). Writes and reads are each individually blocked whenever they'd be illegal (writing while full, reading while empty), and the design correctly supports asserting both `wr_en` and `rd_en` in the same cycle — both go through simultaneously without changing the entry count, except when the FIFO is already completely full (only the read goes through, freeing a slot) or completely empty (only the write goes through). One thing worth knowing before writing tests against it: `dout` is a combinational “peek” at whatever the read pointer currently points to, not a registered output — it updates immediately, and if read while `empty=1` it shows stale leftover data, so any check must gate on `empty` before trusting `dout`.

Key Features

- Count-register-based full/empty — avoids the `wr_ptr==rd_ptr` ambiguity between “just reset” and “completely full.”
- Correctly handles simultaneous read+write in all three fullness states (full, empty, partially-filled), confirmed via targeted simulation.
- Combinational “peek” read port — no separate output register, keeping the design to exactly the four registers specified (memory, both pointers, count).
- Parameterized WIDTH and DEPTH.

Interface Table

Port	Direction	Width	Clock Domain	Active Level	Description
------	-----------	-------	--------------	--------------	-------------

Port	Direction	Width	Clock Domain	Active Level	Description
clk	input	1	— (source)	rising-edge	System clock. All sequential logic (memory, both pointers, count) updates on the rising edge.
rst_n	input	1	clk	active-low	Asynchronous reset. Forces <code>wr_ptr</code> , <code>rd_ptr</code> , and <code>count</code> to 0. Memory not explicitly cleared — <code>count=0</code> marks all entries invalid regardless of stale data.
wr_en	input	1	clk	active-high	Write request. Internally qualified by <code>!full</code> — a write only happens if there's room; otherwise silently dropped.
rd_en	input	1	clk	active-high	Read request. Internally qualified by <code>!empty</code> — a read only happens if a valid entry exists; otherwise silently dropped.
din	input	WIDTH (8)	clk	— (data)	Write data. Captured into memory only on a cycle where a write actually proceeds.
dout	output	WIDTH (8)	clk (combinational)	— (data)	Combinational peek at <code>mem[rd_ptr]</code> . Only meaningful when <code>empty=0</code> .
full	output	1	clk (combinational)	active-high	Asserted when <code>count==DEPTH</code> . Writes blocked while HIGH.
empty	output	1	clk (combinational)	active-high	Asserted when <code>count==0</code> . Reads blocked while HIGH.

Microarchitecture

Unlike the two FSM-based designs, this is a datapath-centric design — correctness lives in pointer/count bookkeeping, not state transitions. Memory array, write pointer, read pointer, and count are all updated in ONE `always_ff` block. Both `wr_en` and `rd_en` are individually qualified by `!full` / `!empty` BEFORE the case statement decides which operation applies — this single design choice correctly handles every combination automatically, including the trickier full/empty simultaneous-access cases (see Corner Cases).

FSM Specification

This design has no explicit FSM — its behavior is driven entirely by pointer and count arithmetic. The closest analog is the control truth table below: which combination of (write request, read request), each already qualified by the current full/empty status, determines what happens each cycle.

wr_en && !full	rd_en && !empty	Action
0	0	No operation — pointers and count unchanged.
1	0	Write only — <code>mem[wr_ptr] <= din</code> ; <code>wr_ptr</code> advances (with wraparound); count increments.
0	1	Read only — <code>rd_ptr</code> advances (with wraparound); count decrements.
1	1	Simultaneous read + write — both pointers advance; count UNCHANGED.

Timing

Actual simulation trace (Icarus Verilog v12.0) — fill to full, blocked write, simultaneous read+write (while full, then partially filled), drain, blocked read, simultaneous read+write while empty:

Cycle	Phase	wr	rd	din	dout	count	full	empty
1–8	fill	1	0	10..17	10	1..8	0 (1 at c8)	0
9	write-while-full	1	0	ff	10	8	1	0
10	simul-RW-while-full	1	1	a0	11	7	0	0
11	simul-RW-normal	1	1	a1	12	7	0	0
13–19	drain	0	1	—	13..a1,11	6..0	0	0 (1 at c19)
20	read-while-empty	0	1	—	11	0	0	1
21	simul-RW-while-empty	1	1	b0	b0	1	0	0
22	final-drain	0	1	—	12	0	0	1

Full trace and verification checklist in *experiments/sync_fifo_smoke_Log.md*. All results — including the poison values 0xFF (blocked write) and 0xA0 (blocked simultaneous write while full) never appearing in any subsequent read — matched hand-predicted behavior exactly.

Corner Cases

#	Corner Case	Expected Behavior
1	Write while full	Blocked — no data corruption, count never exceeds DEPTH. Confirmed (poison value never appears in any later read).
2	Read while empty	Blocked — no count underflow. Confirmed.
3	Simultaneous read+write — full / empty / partially-filled sub-cases	count behaves correctly in all three: unchanged when partially filled, decrement-only when full, increment-only when empty. All confirmed (ISSUE-D6-01).
4	Pointer wraparound exactly at the DEPTH boundary	Must not skip or duplicate a memory slot. Confirmed by the self-checking multi-wrap regression (<i>sync_fifo_multiwrap_tb</i> , ISSUE-D7-02): 3 consecutive fill-then-drain cycles (24 writes / 24 reads, both pointers wrapping multiple times), every read matched against a software reference queue — PASS.

Verification Plan

Refined across Days 9–11 from an original 5-feature list (Day 3) — one over-broad feature was split in two, and a missing symmetric feature (empty/read-blocking) was found and added, bringing the total to 8.

Feature	Req-ID	Cat.	Checker	Stimulus	Pri.	Coverage Item
reset_initializes_fifo_to_empty	RST_001	RST	Expected value	Directed	P0	Coverpoint (count,full,empty) post-reset; bin {0,0,1}
write_increments_count_and_stores_data_correctly	FUNC_001	FUNC	Scoreboard	Constrained-random	P0	Coverpoint count during writes; bins 1 through DEPTH
read_returns_data	FUNC_002	FUNC	Scoreboard	Constrained-	P0	Coverpoint count at

Feature	Req-ID	Cat.	Checker	Stimulus	Pri.	Coverage Item
_in_fifo_order				random		each read; bins 1 through DEPTH
full_flag_asserts_when_count_equals_depth	CORNER_001	CORNER	Expected value	Directed	P0	Coverpoint count, bin full={DEPTH}
write_blocked_when_full	NEG_001	NEG	Assertion	Directed	P0	Cross wr_en × full; bin {1,1}
empty_flag_asserts_when_count_equals_zero	CORNER_002	CORNER	Expected value	Directed	P0	Coverpoint count, bin empty={0}
read_blocked_when_empty	NEG_002	NEG	Assertion	Directed	P0	Cross rd_en × empty; bin {1,1}
count_unchanged_on_simultaneous_read_write	CORNER_003	CORNER	Assertion	Directed (full/empty/normal sub-cases)	P1	Cross wr_en × rd_en × {full,empty,mid}; 3 bins

count coverage plan (5 bins, not the minimal 3): empty={0}, almost_empty={1}, mid={2..6}, almost_full={7}, full={8}. The two boundary-adjacent bins exist specifically because ISSUE-D5-02 (Traffic Light) was itself an off-by-one error sitting next to a boundary, not at it.

RTL Code

```
// =====
`timescale 1ns/1ps
// Module      : sync_fifo
// Description : Synchronous FIFO (single clock domain for both read and
//               write sides - Learning Topic 6.1). Memory array, write
//               pointer, read pointer, and the count register are all
//               updated in ONE always_ff block, per the assignment.
//               full/empty are derived COMBINATIONALLY from count
//               (Learning Topic 6.4 - raw wr_ptr==rd_ptr comparison alone
//               is ambiguous between "just reset" and "completely full",
//               so a count register is used instead, which is the simpler
//               of the two standard approaches).
// =====
```

```
module sync_fifo #(
    parameter int WIDTH = 8,
    parameter int DEPTH = 8
) (
    input logic clk,
    input logic rst_n,           // active-low asynchronous reset
    input logic wr_en,
    input logic rd_en,
    input logic [WIDTH-1:0] din,
    output logic [WIDTH-1:0] dout,
    output logic full,
    output logic empty
);

    // Pointer width only needs to address DEPTH locations (0..DEPTH-1).
    localparam int PTR_WIDTH = $clog2(DEPTH);
    // Count must represent 0..DEPTH inclusive (DEPTH+1 possible values),
    // which needs one more bit than the pointers themselves - e.g. for
    // DEPTH=8, pointers are 3 bits (0-7) but count needs 4 bits (0-8).
    localparam int CNT_WIDTH = $clog2(DEPTH + 1);
```

```

logic [WIDTH-1:0] mem [0:DEPTH-1];
logic [PTR_WIDTH-1:0] wr_ptr, rd_ptr;
logic [CNT_WIDTH-1:0] count;

// full/empty - combinational, derived from count alone (not from
// comparing wr_ptr to rd_ptr, which cannot distinguish full from empty
// on its own - Learning Topic 6.4).
assign full = (count == DEPTH);
assign empty = (count == 0);

// Read data - combinational "peek" at the current front of the queue.
// Design decision (documented explicitly, not left implicit - Day 7
// principle): dout always shows mem[rd_ptr], regardless of rd_en. When
// rd_en pulses (and the FIFO isn't empty), rd_ptr advances on the next
// clock edge and dout will then show the NEXT entry. This keeps the
// design to exactly the four registers the assignment specifies
// (mem, wr_ptr, rd_ptr, count) with no separate output register.
assign dout = mem[rd_ptr];

// -----
// Memory array, write pointer, read pointer, and count - all in
// ONE always_ff block, per the assignment.
//
// Both wr_en and rd_en are individually qualified by !full / !empty
// BEFORE deciding which case applies. This is what correctly handles
// every combination automatically, including the trickier ones:
// - write while full      -> blocked (data would be lost)
// - read while empty      -> blocked (undefined data)
// - simultaneous read+write -> count unchanged, both proceed
// - simultaneous read+write,
//   but FIFO is full      -> only the read proceeds (frees a
//                           slot; the write is still blocked
//                           since there was no room BEFORE
//                           this cycle's read takes effect)
// - simultaneous read+write,
//   but FIFO is empty     -> only the write proceeds
// -----
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        wr_ptr <= '0;
        rd_ptr <= '0;
        count <= '0;
    end else begin
        case ({wr_en && !full, rd_en && !empty})
            2'b10: begin // write only
                mem[wr_ptr] <= din;
                wr_ptr <= (wr_ptr == DEPTH - 1) ? '0 : wr_ptr + 1'b1;
                count <= count + 1'b1;
            end
            2'b01: begin // read only
                rd_ptr <= (rd_ptr == DEPTH - 1) ? '0 : rd_ptr + 1'b1;
                count <= count - 1'b1;
            end
            2'b11: begin // simultaneous read + write - count unchanged
                mem[wr_ptr] <= din;
                wr_ptr <= (wr_ptr == DEPTH - 1) ? '0 : wr_ptr + 1'b1;
                rd_ptr <= (rd_ptr == DEPTH - 1) ? '0 : rd_ptr + 1'b1;
            end
            default: begin // 2'b00 - no valid operation this cycle
                // explicit empty branch (not left as an implicit
                // "do nothing") so nothing is inferred as a latch
            end
        endcase
    end
end

```

endmodule